Université Grenoble Alpes
L3 Informatique

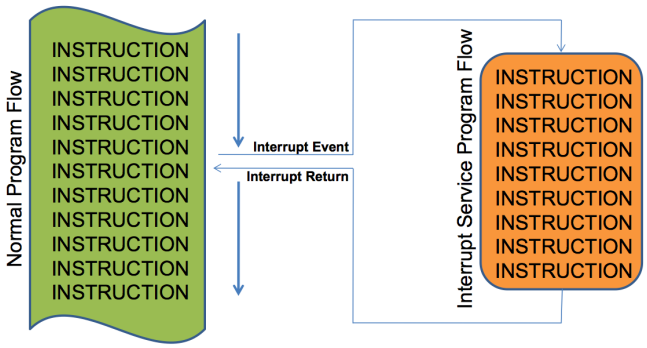
Année 2026-2027

Architectures logicielles et matérielles
Semaine 5 TD 2

Exercice 1.

Nous avons étudié la façon dont le processeur gère le départ en interruption et le retour d'interruption. Rappelons le lien avec le point de vue du développeur logiciel.

Q1. Une vue schématique assez usuelle du flot d'exécution en présence d'une interruption a été rappelée au cours 4 (slide 22)



Expliquer ce que représentent les deux blocs d'instructions sur ce schéma, et où sont schématisés le départ en interruption et le retour d'interruption.
Expliquer le lien conceptuel entre les actions du processeur vues au TD1 et cette vue "développeur" ici : où se place ici le rôle des transitions de l'automate d'interprétation servant à gérer les interruptions, et comment ? où se place ici le rôle de l'instruction *rti*, et comment ?

Q2. Expliquer la différence au niveau du processeur entre le traitement d'un appel de **fonction** "ordinaire" et la gestion d'une **interruption**. Dans chaque cas : y a-t-il appel explicite dans le programme ? comment la nouvelle valeur de PC, et l'adresse de retour, sont-elles connues ? comment se fera le retour en fin d'exécution ?

Q3. Pour mettre en oeuvre une solution logicielle exploitant les **interruptions** déclenchées par un périphérique :
- qu'est-ce que le programmeur **doit prévoir d'ajouter** dans son application (outre le fait d'avoir correctement configuré le périphérique) ?
- et **quelles actions** doivent y être réalisées ?

Exercice 2.

Q1. Quel est le rôle d'un **timer** ? (tel que le *systick* dans le microcontrôleur de la carte STM32F4)

NB. Pour la suite, on rappelle que la fonction gestionnaire d'interruptions associée au *systick* dans la bibliothèque libopencm3 porte le nom *sys_tick_handler.* Et que les interruptions du *systick* ne sont pas gérées par le NVIC (Nested Vectored Interrupt Controller), et n'ont pas non plus besoin d'être annulées.

Q2. Nous rappelons en annexe le code fourni pour la petite application étudiée en semaine 3 (allumage de leds). Observer la boucle while de la fonction *main* : **que réalise-t-elle** et quel sera l'**effet** observable ?

Q3. Notre objectif ici sera de modifier ce code afin d'**exécuter la fonction *plusplus* périodiquement** (à la fréquence du systick) : **pourquoi** cela permettra-t-il d'obtenir une évolution périodique de l'allumage des leds ?

Q4. Que faut-il ajouter dans ce code pour obtenir que *plusplus* soit exécutée comme attendu ? Ecrire le **code** correspondant.

Q5. Nous compilons et chargeons l'application ainsi modifiée. Nous plaçons un breakpoint sur la fonction *plusplus* et nous faisons continuer l'exécution. Lorsqu'elle atteint ce point d'arrêt, nous observons **la pile des appels de fonctions**. L'observation réalisée est la suivante :

```
(gdb) b plusplus
Breakpoint 1 at 0x8000204: file firmware.c, line 35.

(gdb) c
Continuing.
halted: PC: 0x080001f4

Breakpoint 1, plusplus () at firmware.c:35
3     n= n+delta;

(gdb) backtrace
#0  plusplus () at firmware.c:35
#1  0x..... in sys_tick_handler () at firmware.c:41
#2  <signal handler called>
#3  0x x..... in write_leds_int (intval=0) at bp_leds.c:15
#4  0x x..... in main () at firmware.c:103

(gdb)...
```

Expliquer cette observation.

Exercice 3.

L'objectif maintenant sera d'observer l'effet de telles modifications dans la **table des vecteurs d'interruptions**.

Q1. Rappeler la définition de la *table des **vecteurs d'interruptions***, et expliquer soigneusement **votre compréhension** de son utilité : elle est utilisée dans quels cas, comment,… ?

Q2. Nous allons ci-dessous analyser des expérimentations, faites **manuellement** dans gdb, dans le but d'obtenir un résultat similaire à ce qui a été fait à l'exercice 2.

Nous considérons ici à nouveau le programme tel que fourni en annexe, sans définition du handler *sys_tick_handler.*
Les manipulations qui suivent ont été réalisées sur la carte STM32F4 après chargement de l'exécutable en RAM[1] afin de pouvoir modifier manuellement la table des vecteurs d'interruptions.
Observer les manipulations ci-dessous et **répondre aux questions**.

Affichons l'adresse de la **table des vecteur d'interruptions** :

```
(gdb) print /x &vector_table
$1 = 0x20000000
```

Affichons son contenu :

```
(gdb) print vector_table
$2 = {initial_sp_value = 0x20020000, reset = 0x20000b2d <reset_handler>,
      nmi = 0x20000b2b <null_handler>,
      hard_fault = 0x20000b29 <blocking_handler>,
      memory_manage_fault = 0x20000b29 <blocking_handler>,
      bus_fault = 0x20000b29 <blocking_handler>,
      usage_fault = 0x20000b29 <blocking_handler>,
      reserved_x001c = {0x0, 0x0, 0x0, 0x0},
      sv_call = 0x20000b2b <null_handler>,
      debug_monitor = 0x20000b2b <null_handler>,
      reserved_x0034 = 0x0, pend_sv = 0x20000b2b <null_handler>,
      systick = 0x20000b2b <null_handler>, irq = {
      0x20000b29 <blocking_handler> <repeats 91 times>}}
```

[1] Lors des TP des premières séances, votre code était chargé en ROM.

=> Quelle est **l'adresse** de l'ISR associée à *systick* et quel est le **nom** de cette fonction ?

Observons la définition de cette fonction :

```
(gdb) x /i 0x20000b2b
   0x20000b2b <null_handler>:     bx  lr
```

=> **Que fait-elle** ?

Vérifions néanmoins que notre *systick* est bien activé, et que cette ISR est bien appelée :

```
(gdb) b null_handler
Breakpoint 3 at 0x20000b2a: file ../../cm3/vector.c, line 110.

(gdb) c
Continuing.
Breakpoint 3, null_handler () at ../../cm3/vector.c:110

(gdb) backtrace
#0  null_handler () at ../../cm3/vector.c:110
#1  <signal handler called>
#2  0x20000398 in write_leds_int (intval=0) at bp_leds.c:15
#3  0x20000338 in main () at firmware.c:100
```

=> **Expliquer** l'observation ci-dessus.

Modifions maintenant le contenu de la table des vecteurs d'interruptions et observons l'effet de cette modification :

```
(gdb) print /x &vector_table.systick
$3 = 0x2000003c

(gdb) set *0x2000003c=&plusplus+1

(gdb) print vector_table
$2 = {initial_sp_value = 0x20020000, reset = 0x20000b2d <reset_handler>,
      nmi = 0x20000b2b <null_handler>,
      hard_fault = 0x20000b29 <blocking_handler>,
      memory_manage_fault = 0x20000b29 <blocking_handler>,
      bus_fault = 0x20000b29 <blocking_handler>,
      usage_fault = 0x20000b29 <blocking_handler>,
      reserved_x001c = {0x0, 0x0, 0x0, 0x0},
      sv_call = 0x20000b2b <null_handler>,
      debug_monitor = 0x20000b2b <null_handler>,
      reserved_x0034 = 0x0, pend_sv = 0x20000b2b <null_handler>,
      systick = 0x20000201 <plusplus>, irq = {
      0x20000b29 <blocking_handler> <repeats 91 times>}}

(gdb) b plusplus
Breakpoint 2 at 0x20000204: file firmware.c, line 35.

(gdb) c
Continuing.
halted: PC: 0x200001f4
Breakpoint 2, plusplus () at firmware.c:35
35        n= n+delta;

(gdb) back
#0  plusplus () at firmware.c:35
#1  <signal handler called>
#2  0x20000396 in write_leds_int (intval=0) at bp_leds.c:15
#3  0x20000338 in main () at firmware.c:101
```

=> **Expliquer** la manipulation et l'observation ci-dessus.

Q3. Conclusion : pourquoi cette manipulation réalisée manuellement est-elle équivalente à la modification logicielle réalisée à l'exercice 2 ? Et que vous a-t-elle permis d'observer ?

Note. Le Cortex-M4 est un microcontrôleur basé sur une architecture Armv7, qui n'a pas d'instruction spécifique *rti*. Le retour d'interruption utilise une instruction `bx lr`
Dans le cas où le processeur est en mode interruption, cette instruction charge une valeur spéciale dans le PC (de la forme 0xFXXXXXXX), ce qui provoque un traitement de type retour d'interruption.

==========

Annexe

```
#define RELOAD_VAl_700MS 0xffffff
#define RELOAD_VAl_350MS 0x8fffff

uint16_t n=0;

int delta=1;

void plusplus () {   // incrementation de n
  n = n+delta;
}

void systick_config (uint32_t reload_val) {   // configuration systick
   systick_set_clocksource(STK_CSR_CLKSOURCE_AHB_DIV8);
   systick_set_reload(reload_val);
   systick_interrupt_enable();
   systick_counter_enable();
}

void config_leds_pins () {       // configuration leds
   gpio_mode_setup(GPIOD,GPIO_MODE_OUTPUT,GPIO_PUPD_NONE,LEDS_MASK);
}

void write_leds_int (uint16_t intval) {    // allumage leds selon la
   ...                                     // valeur de intval
   ...
}

int main (void) {
   coldstart();
   config_leds_pins();
   systick_config(RELOAD_VAl_350MS);
   while (1) {
      write_leds_int(n%12);
   }
   return 0;
}
```